

MILESTONE- 4

1. Executive Summary

"The Mystical Oracle" has successfully evolved into a fully integrated, multi-modal Conversational AI application. The platform seamlessly bridges a React-based frontend with a FastAPI backend, powered by custom deep learning segmentation models and ultra-low latency Large Language Models (LLMs). This report details the transition from classical image processing to robust deep learning, the engineering of stateful conversational memory, persistent database storage, and full-stack integration that enables dynamic, multi-turn user interactions.

The system supports both Palmistry and Tarot-based interactions, allowing users to upload palm images or interact with Tarot readings and subsequently engage in contextual conversations with the AI. User sessions and conversation histories are persistently stored, allowing previous conversations to be reopened and continued from exactly where the user left off. Users can also delete stored conversations when they are no longer required.

This combination of sophisticated computer vision, generative AI, persistent data management, and user-centric design creates a uniquely accessible and engaging mystical experience.

1.1 System Workflow

The Mystical Oracle operates as a seamless loop: users authenticate via a secure OTP, select their method of divination (Palmistry or Tarot), and upload input (image or question). The backend processes this through Computer Vision models or Tarot dataset lookups, generates an AI-powered reading through LLM-driven prompt engineering, and stores the complete session in the database.

When a user returns to a previous session, the stored conversation history is retrieved and restored, allowing the user to continue the conversation from the exact point where it was previously stopped. Users can also delete individual stored conversations through the database management interface.

2. Backend: Computer Vision & AI Pipelines

The core challenge of this application was accurately extracting physical features from user-uploaded images and translating that data into a format an LLM could understand.

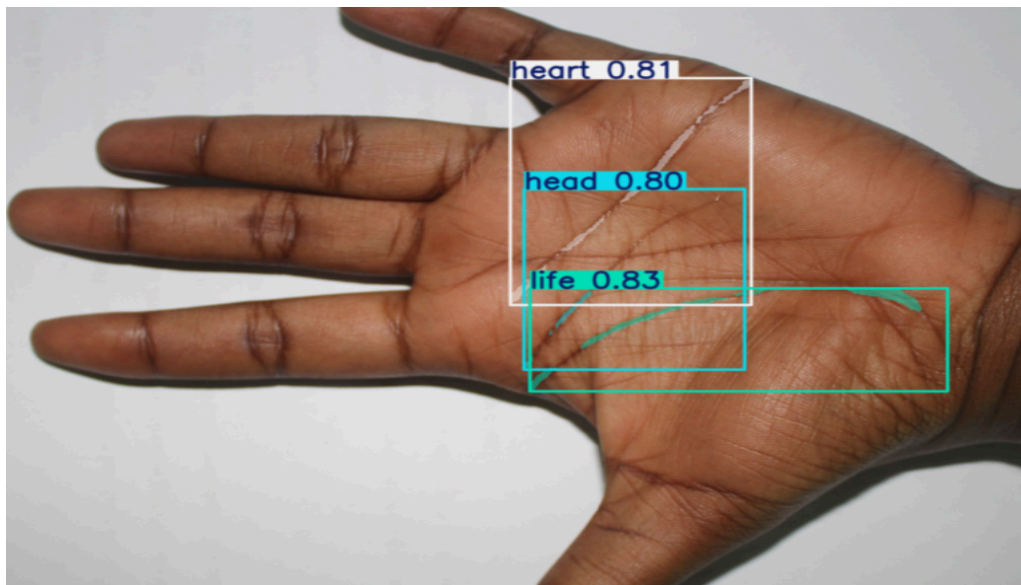
2.1 The Deep Learning Upgrade (YOLO11-Seg)

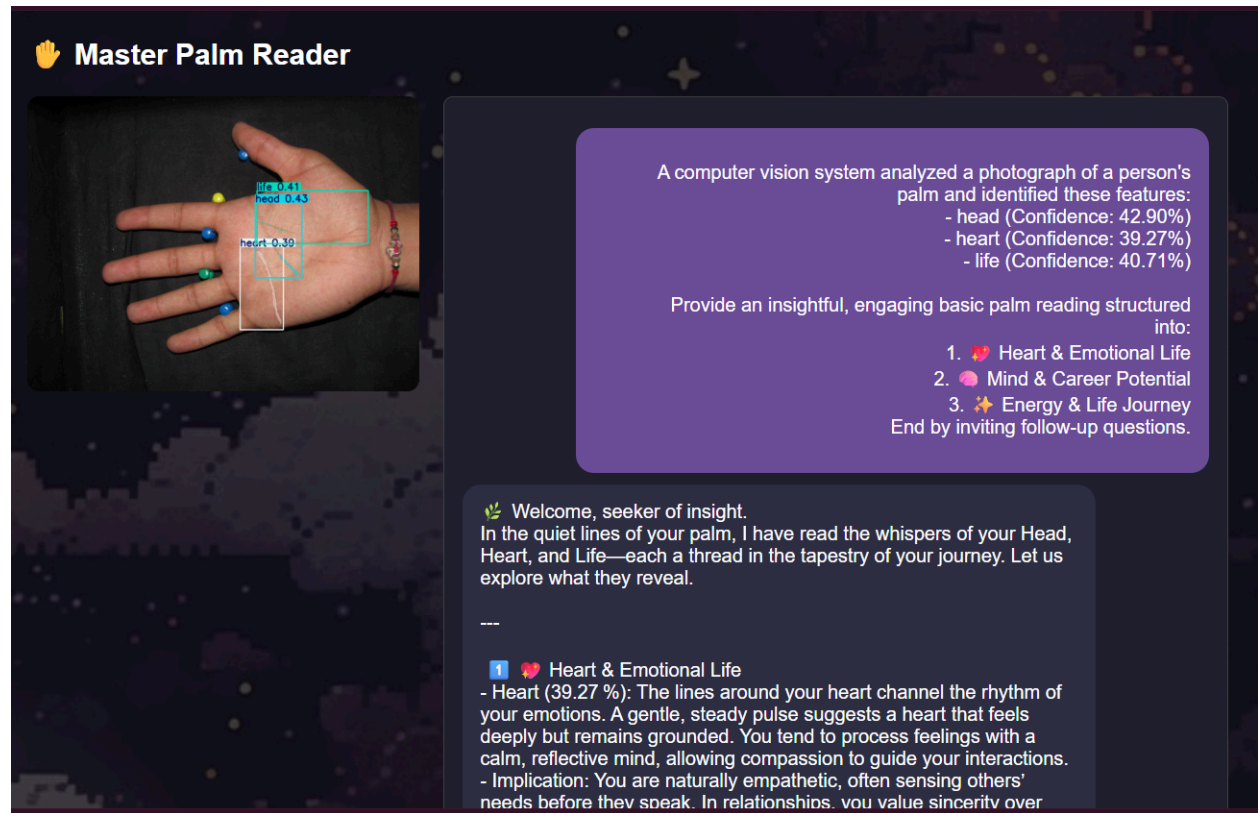
Classical Limitations: The initial pipeline utilized MediaPipe and OpenCV for thresholding and skeletonization, but this proved brittle and inconsistent across diverse lighting conditions and skin tones.

Model Selection: We successfully resolved this by pivoting to a deep learning methodology, specifically deploying a YOLO11-seg (Segmentation) model.

Dataset & Training: We utilized Roboflow to manage and annotate a custom dataset named `palms-lines-4`, consisting of 258 training images.

Performance Metrics: The model was trained using PyTorch and the AdamW optimizer for up to 100 epochs, with an early stopping mechanism triggering at epoch 42. Validation results were highly successful, achieving a mAP50 of 0.995 for the Head Line, 0.862 for the Heart Line, and 0.832 for the Life Line.





2.2 Data Preprocessing (Tarot Engine)

EDA & Cleaning: To support the Tarot module, we ingested the [lsind18/tarot-json](#) dataset and performed Exploratory Data Analysis (EDA) using Pandas. Missing metadata fields were identified and imputed with clean default values to ensure seamless downstream processing.

Visualization: We utilized Seaborn and Matplotlib to map the distribution of Major and Minor Arcana suits. The engine dynamically applies randomized logic for Upright versus Reversed orientations and renders the cards using PIL.

3. Backend: Generative AI & Stateful Memory

Generating a one-off response is easy; maintaining context across a session requires a specialized architecture.

3.1 Dynamic Prompt Engineering

Instead of relying on static labels, we engineered a pipeline that extracts numerical outputs from the YOLO11 model—specifically the class labels and confidence scores, such as `life: 83.15%`—and formats them directly into a structured context prompt.

We utilize the Groq API to access the `llama-3.3-70b-versatile` model, but as it eventually got taken down so i used `openai/gpt-oss-20b`

instructing it through strict prompt engineering to adopt the persona of a Master Palm Reader.

3.2 Stateful Conversation Architecture

The Challenge: Initially, the LLM suffered from context loss, generating generic advice during follow-up questions because it "forgot" the initial vision scan or drawn cards.

The Resolution: We engineered a Stateful Conversation History buffer. By utilizing Python lists and dictionaries to map roles such as `system`, `user`, and `assistant`, the application retains the entire context of the session.

Execution: The conversation history is passed back to the Groq API on every turn, allowing the AI to provide empathetic, deeply contextual, and ethical responses without losing sight of the initial visual detections, Tarot cards, or previous discussion.

The stateful architecture is further supported by persistent database storage, allowing this conversation context to survive beyond a single browser session.

3.3 AI Persona & Ethical Consistency

To ensure a consistent and mystical experience, the AI is constrained by strict system prompts. It is instructed to adopt the persona of a wise, empathetic reader, avoiding clinical or robotic phrasing. This ensures that the advice provided is consistently supportive and adheres to the user's need for a meaningful, interpretive experience.

4. Frontend: User Interface & Experience

The backend complexities are entirely abstracted away from the user through a highly interactive React frontend.

4.1 Authentication & Security

The application features a robust **AuthScreen** component that supports registration via email or international mobile numbers, utilizing a comprehensive **COUNTRY_CODES** mapping.

Security is handled through an OTP (One-Time Password) authentication mechanism, reducing dependency on traditional password-based authentication.

The screenshots display the user interface of 'The Mystical Oracle' application, which features a dark, mystical theme with a starry background.

Top Left Screenshot (Login Screen): The screen is titled 'The Mystical Oracle'. It has two tabs: 'Sign In' (active) and 'New Seeker (Sign Up)'. A message states: 'Account created successfully! Please sign in.' The 'Sign In' form includes a 'Username' field with the value 'sonam', a 'Secret Cipher (Password)' field with the placeholder 'Enter password...', a 'Forgot Password?' link, and a purple 'Enter Portal' button.

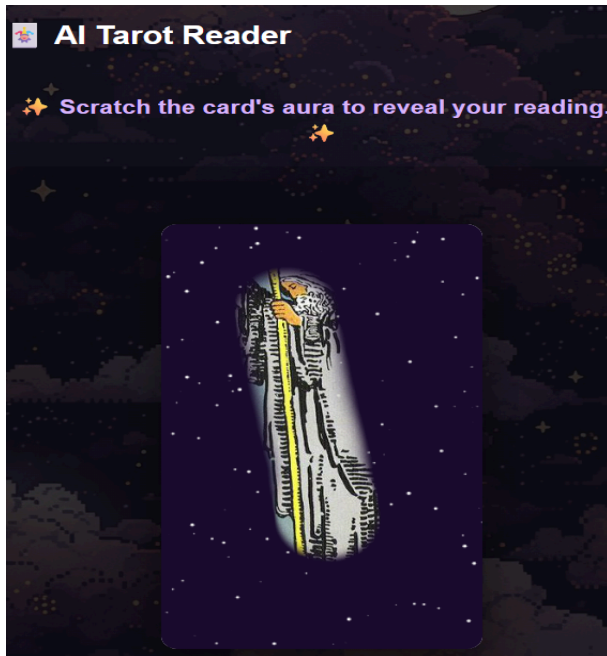
Top Right Screenshot (Recovery Screen): The screen is titled 'The Mystical Oracle'. It has a tab 'Recover Your Cipher'. The text says: 'Enter your username, registered email, or full phone number with country code.' The form includes a text input field with the placeholder 'Username / Email / Phone...', a purple 'Summon OTP' button, and a link 'Back to Sign In'.

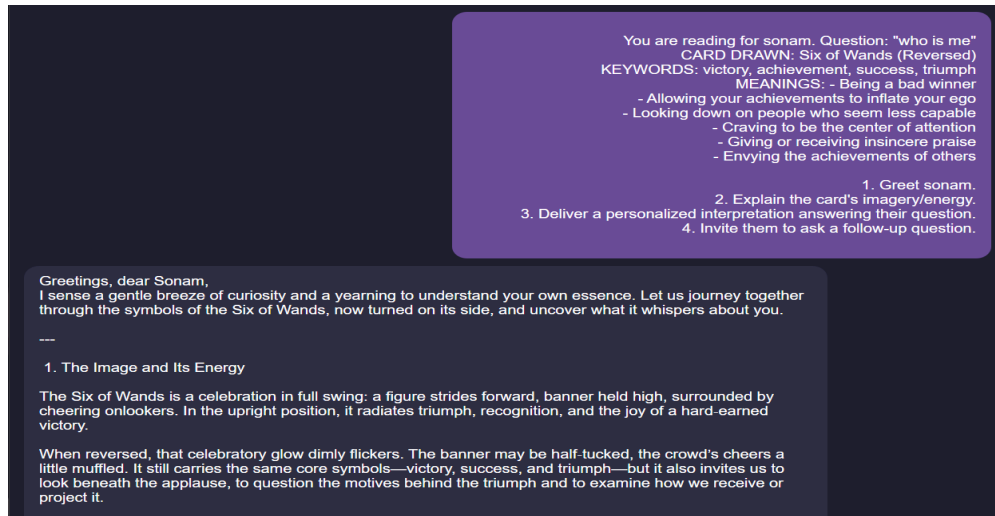
Bottom Screenshot (Registration Screen): The screen is titled 'The Mystical Oracle'. It has two tabs: 'Sign In' and 'New Seeker (Sign Up)' (active). The 'New Seeker' form includes a 'Choose Username' field with the value 'sonam', an 'Authentication Method' dropdown menu set to 'Mobile Phone', a 'Phone Number' section with a country code dropdown set to 'IN +91 (India)' and a phone number field with the value '9175110079', a 'Create Password' field with masked characters '*****', a 'Confirm Password' field with masked characters '*****', and a green 'Create Account' button.

4.2 Interactive UI Elements

Tarot Scratch-Off

The frontend utilizes an `<AuraReveal1/>` component, requiring users to interactively scratch the image/base64 data to reveal their drawn card, which subsequently activates the chat interface.





Live Chat Loop

The frontend seamlessly maps to the backend's stateful memory architecture. A custom **ChatBox** component manages the UI for the continuous, multi-turn dialogue, displaying system loading states such as "The spirits are typing..." while the Groq API processes the inference.

Persistent Conversation Management

The frontend communicates with the backend to store and retrieve conversation sessions. This enables users to access previous readings instead of losing their conversations after closing the application.

4.3 Database Storage & Conversation Management

A dedicated database storage system has been integrated into The Mystical Oracle to provide persistent storage for user conversations and reading sessions.

Unlike temporary browser-only storage, database persistence allows conversations to remain available even after the user closes the browser or returns to the application at a later time.

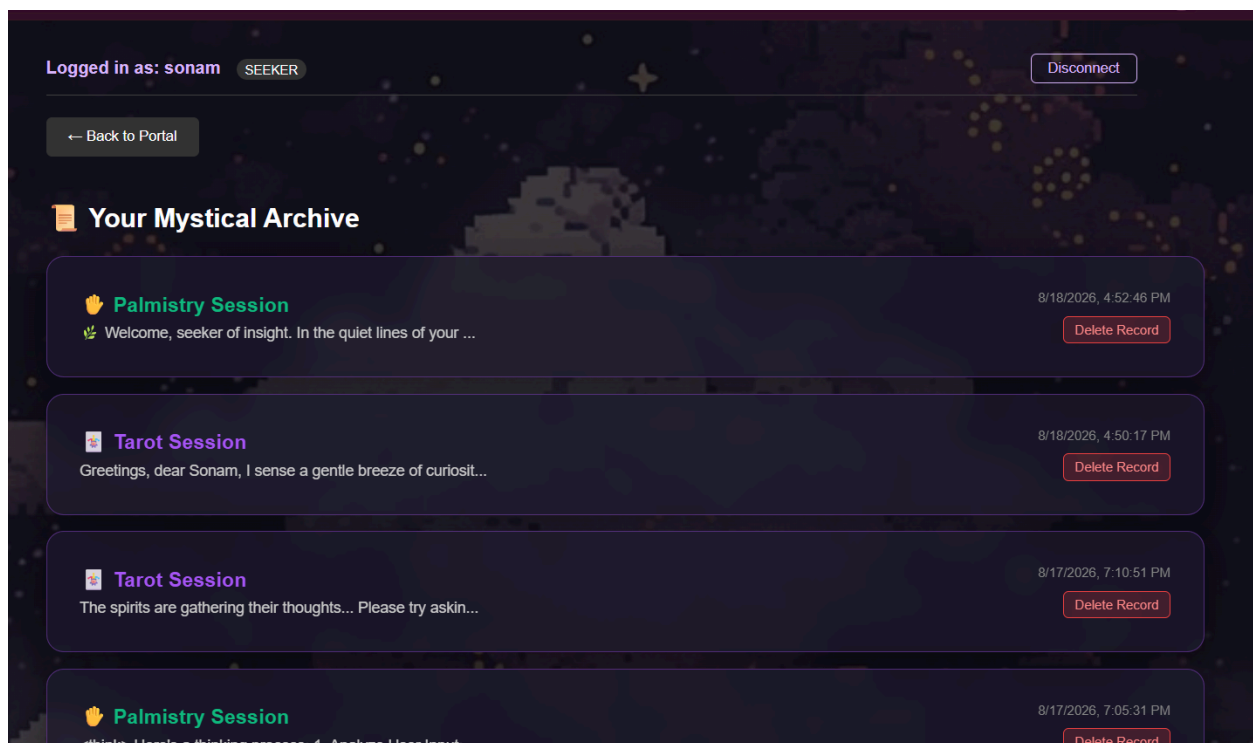
The database stores the relevant conversation information, including previous user messages, AI responses, and session context. When a user opens an existing conversation, the stored history is retrieved and reconstructed so that the AI can continue the interaction from the exact point where the previous conversation ended.

This creates a continuous conversational experience rather than treating every interaction as a new session.

Key Database Features

- **Persistent Storage:** Conversation histories remain stored beyond the current browser session.
- **Conversation Retrieval:** Previously saved readings can be reopened by the user.
- **Context Preservation:** Previous messages and AI responses are retained, allowing the conversation to continue with its existing context.
- **Resume From Previous State:** Users can return to an earlier conversation and continue from exactly where they left off.
- **Conversation Deletion:** A dedicated Delete option allows users to permanently remove unwanted or outdated conversations from their stored history.
- **Session Organization:** Individual readings/conversations can be maintained as separate sessions, making it easier to manage multiple Palmistry or Tarot interactions.

The database therefore acts as the persistent layer between the frontend interface and the stateful AI conversation system.



4.4 Security Philosophy

We prioritize user security through a passwordless authentication model. By utilizing an OTP-based system, we reduce the risk of credential leakage and remove the cognitive burden of managing and remembering complex passwords, providing a streamlined and secure entry into the platform.

5. Key Architectural Highlights

Robust API Resilience

The implementation of a rotating API key strategy ensures high availability and robust quota management for the AI endpoints.

Intelligent CV-LLM Bridging

The system seamlessly converts computer vision segmentation confidence scores into structured, natural language prompts, enabling highly personalized and specific readings.

Stateful Memory Architecture

The dedicated conversation history buffer maintains session context, ensuring consistent and empathetic multi-turn dialogue throughout the user's journey.

Persistent Database Architecture

Database-backed conversation storage allows users to preserve, retrieve, resume, and delete previous sessions. This extends the stateful architecture beyond a single browser session and provides a reliable mechanism for long-term conversation persistence.

UX-Focused Interaction Design

The application features highly engaging interactive elements, such as the `<AuraReveal/>` scratch-off component, paired with secure authentication flows and persistent conversation management.

6. Future Roadmap

6.1 Real-Time Notifications

Integrating professional services such as Twilio or SendGrid to replace simulated OTP alerts with genuine user notification pipelines.

6.2 Advanced Computer Vision

Expanding YOLO capabilities to analyze additional palm features, lines, textures, and other visual characteristics for higher-fidelity readings.

6.3 Enhanced Conversation Management

Future versions can introduce advanced conversation organization features such as search, filtering, conversation titles, timestamps, favorites, and improved session management.

6.4 Scalability & Deployment

Further optimization of the backend and database architecture can support larger numbers of concurrent users and enable reliable cloud deployment.

7. Technical Challenges & Solutions

Inference Latency

Balancing the processing speed of YOLO Computer Vision models with LLM response times to maintain a fluid user experience.

Backend Scalability

Refactoring the monolithic structure into an APIRouter-based architecture to support cleaner code, modular development, and easier deployment.

Conversation Context Loss

The initial implementation struggled to maintain context across follow-up questions. This was addressed through a stateful conversation history buffer and persistent database storage, allowing previous messages and AI responses to be retained and restored.

Persistent Data Management

Client-side-only storage was insufficient for long-term conversation persistence. The database storage layer solved this by providing persistent storage, retrieval, and deletion capabilities for user conversations.

Multi-Modal AI Integration

Integrating computer vision outputs, Tarot dataset information, and LLM-generated responses required a structured bridge between different AI components. The final architecture converts CV and Tarot results into contextual information that can be consumed by the LLM to generate personalized responses.